

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Sukesh S 24011101111

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance using experimental performance metrics.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN models using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become important models for computer vision because they automatically learn hierarchical image features. The evolution of CNN architectures has focused on improving classification accuracy while addressing computational complexity and training difficulties.

The progression of CNN architectures includes LeNet-5, AlexNet, VGGNet, GoogleNet and ResNet. Each architecture introduced an important design idea, such as ReLU and dropout, deeper stacks of small convolutions, multi-scale Inception modules, or residual connections.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters and increased depth
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of convolution layers, average pooling layers and fully connected layers. It demonstrated that convolutional networks could learn useful image features automatically.

Advantages

- Small number of parameters.
- Fast training.
- Suitable for OCR and handwritten digit recognition.

6. AlexNet

AlexNet achieved a major breakthrough in the 2012 ImageNet challenge. It introduced ReLU activation, dropout regularization, GPU-based training and data augmentation.

Major innovations

- ReLU activation.
- Dropout.
- GPU implementation.
- Data augmentation.

7. VGG16

VGG16 demonstrated that deep networks using repeated small 3×3 convolution filters can provide strong image representations. The model used a simple and uniform convolutional design while increasing network depth.

Advantages

- Strong feature extraction.
- Simple and uniform architecture.
- High classification accuracy.

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. A typical Inception module can combine 1×1 , 3×3 , 5×5 convolutions and pooling operations. The outputs are concatenated to obtain multi-scale feature representations.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- Efficient computation.
- Strong image-classification performance.

10. ResNet

Increasing CNN depth can make optimization difficult. ResNet addresses this using residual or skip connections, which provide a direct path for information and gradients.

Instead of directly learning

$$H(x),$$

the residual block learns

$$F(x) = H(x) - x$$

and produces

$$\text{Output} = F(x) + x.$$

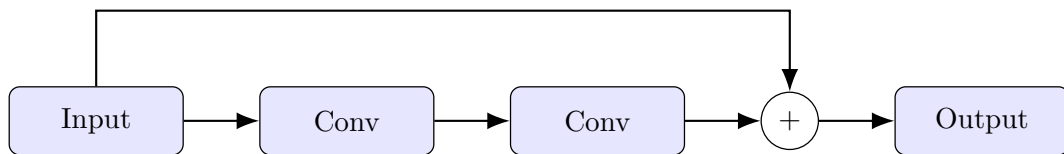


Figure 1: Residual Block in ResNet

Advantages

- Improves gradient flow in deep networks.
- Enables very deep neural networks.
- Helps optimization and convergence.
- Provides strong image-classification performance.

11. Dilated Convolution

Dilated convolution, also called atrous convolution, increases the receptive field without proportionally increasing the number of kernel parameters. The dilation rate determines the spacing between kernel elements.

For standard convolution,

$$D = 1,$$

whereas for dilated convolution,

$$D > 1.$$

Applications

- Semantic segmentation.
- Medical image analysis.
- Satellite imagery.
- Object localization.

12. Transpose Convolution

Transpose convolution performs learnable upsampling and can increase the spatial dimensions of feature maps.

Applications

- Image super-resolution.
- Autoencoders.
- GANs.
- Image generation.
- Semantic segmentation.

13. Transfer Learning

Transfer learning uses a pretrained CNN model that has already learned general visual features from a large dataset such as ImageNet. Instead of training all parameters from scratch, a pretrained convolutional base is adapted to the target classification problem.

The workflow used in this experiment is

1. Load a pretrained VGG16 model with ImageNet weights.
2. Remove the original classification head.
3. Freeze the convolutional base.
4. Add Global Average Pooling.
5. Add a Dense layer with ReLU activation.
6. Add a ten-class Softmax output layer.
7. Train the classifier.
8. Unfreeze the last convolution block and fine tune the model.

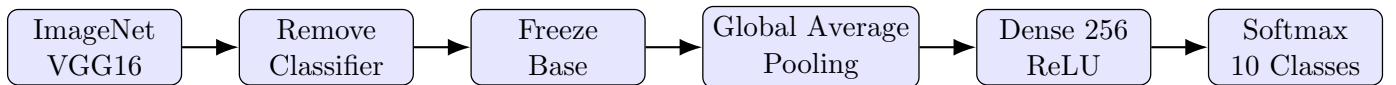


Figure 2: Transfer Learning Workflow Used in the Experiment

14. Dataset

The experiment uses the CIFAR-10 dataset.

- Training images: 50,000.
- Testing images: 10,000.
- Number of classes: 10.
- Image size: $32 \times 32 \times 3$.
- Pixel values are normalized to the range $[0, 1]$.

The classes are:

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

15. Experimental Procedure

Task 1: Dataset Preparation

The CIFAR-10 dataset is loaded using TensorFlow/Keras. The pixel values are converted to floating-point values and divided by 255 so that the input range becomes $[0, 1]$. Ten sample images are displayed with their corresponding class names.



Figure 3: Sample CIFAR-10 images used for dataset inspection.

Inference: The figure shows representative images from the ten CIFAR-10 classes. The images have low spatial resolution (32×32). Making it difficult as compared to larger ones.

Task 2: Transfer Learning

A VGG16 network is loaded with ImageNet pretrained weights and `include_top=False`. The input shape is set to (32, 32, 3). The convolutional base is initially frozen.

The custom classification head is:

VGG16 Base $\rightarrow$ GlobalAveragePooling2D $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dense(10, Softmax).

The VGG16 base contains 14,714,688 non-trainable parameters. The custom classification head contains 133,898 trainable parameters, giving a total of 14,848,586 parameters.

Component	Parameters	Status Initially
VGG16 convolutional base	14,714,688	Frozen
Global Average Pooling	0	Trainable
Dense 256 ReLU	131,328	Trainable
Dense 10 Softmax	2,570	Trainable
Total	14,848,586	—

Task 3: Model Training

The model is compiled using the Adam optimizer with learning rate 0.001, Sparse Categorical Crossentropy loss, and accuracy as the evaluation metric.

The initial training uses:

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10
Loss	Sparse Categorical Crossentropy
Metric	Accuracy

At the end of the initial 10 epochs, the notebook reports:

Training Accuracy = 0.7017, Validation Accuracy = 0.6217.

Inference: The pretrained convolutional base allows the classifier to learn useful informations quickly. The difference between training and validation accuracy indicates that the frozen feature extractor and new classification didn't completely adapt to the CIFAR-10 dataset.

Task 4: Fine Tuning

The last convolutional block of VGG16 is unfrozen while the earlier convolutional blocks remain frozen. The model is recompiled and trained for five additional epochs.

The notebook reports the following comparison:

Stage	Training Accuracy	Validation Accuracy
Before fine tuning	0.7017	0.6217
After fine tuning	0.8526	0.7217

Inference: Fine tuning substantially improves both training and validation accuracy. Validation accuracy increases from 62.17% to 72.17%, showing that adapting the last convolutional block to CIFAR-10 features improves generalization.

Task 5: Model Evaluation

After fine tuning, the model is evaluated on the CIFAR-10 test set.

Metric	Value
Test Loss	0.8814
Test Accuracy	0.7217

The notebook also produces a classification report and confusion matrix.

Class	Precision	Recall	F1-score
Airplane	0.78	0.81	0.80
Automobile	0.72	0.92	0.81
Bird	0.83	0.58	0.68
Cat	0.51	0.61	0.56
Deer	0.63	0.70	0.66
Dog	0.72	0.52	0.60
Frog	0.73	0.80	0.77
Horse	0.69	0.86	0.77
Ship	0.86	0.83	0.84
Truck	0.93	0.57	0.71
Macro Average	0.74	0.72	0.72
Weighted Average	0.74	0.72	0.72

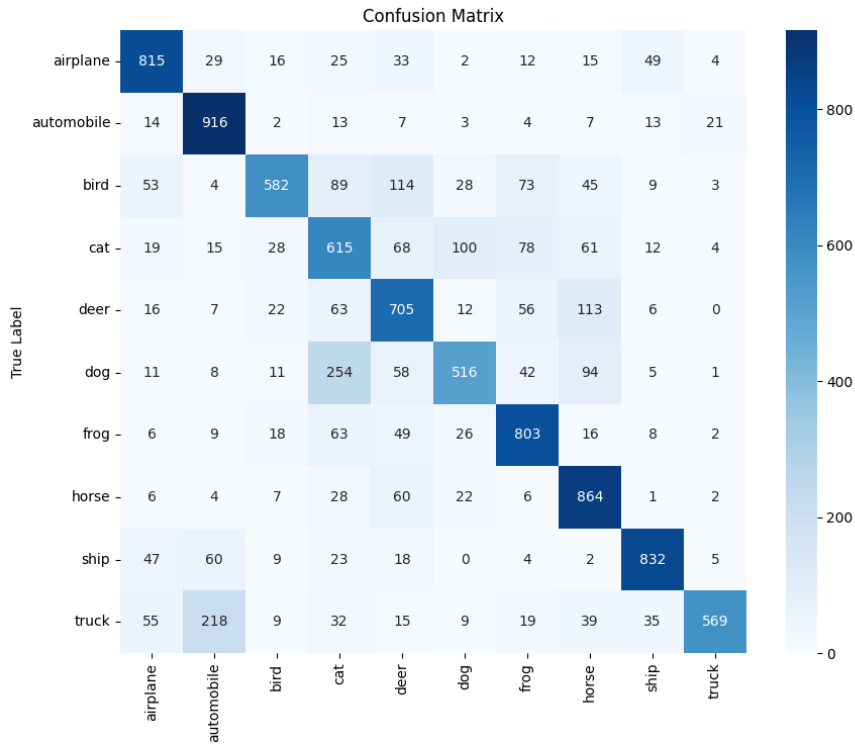


Figure 4: Confusion matrix obtained from the fine-tuned VGG16 model.

Inference: The confusion matrix shows that most of the classes are classified well, especially automobile, airplane, frog, horse and ship. Visually similar images like cat and dog shows lesser positives as it is difficult to find in such small resolution datasets as CIFAR 10.

16. Hyperparameter Study

The notebook first studies learning rate and dense layer size while keeping the remaining settings fixed. Four experiments are performed.

Experiment	Learning Rate	Dense Units	Test Accuracy
1	0.001	128	0.6117
2	0.001	256	0.6134
3	0.0001	128	0.5815
4	0.0001	256	0.5872

The study is then expanded by comparing Adam and SGD as well as fully frozen and partially unfrozen convolutional bases.

Experiment	Optimizer	Frozen Layers	Test Accuracy
5	Adam	All	0.6219
6	Adam	Partial	0.7382
7	SGD	All	0.4934
8	SGD	Partial	0.7124

Experiment 6 is the best-performing configuration:

Learning Rate = 0.001, Dense Units = 256, Optimizer = Adam, Frozen Layers = Partial

Its test accuracy is

73.82%

.

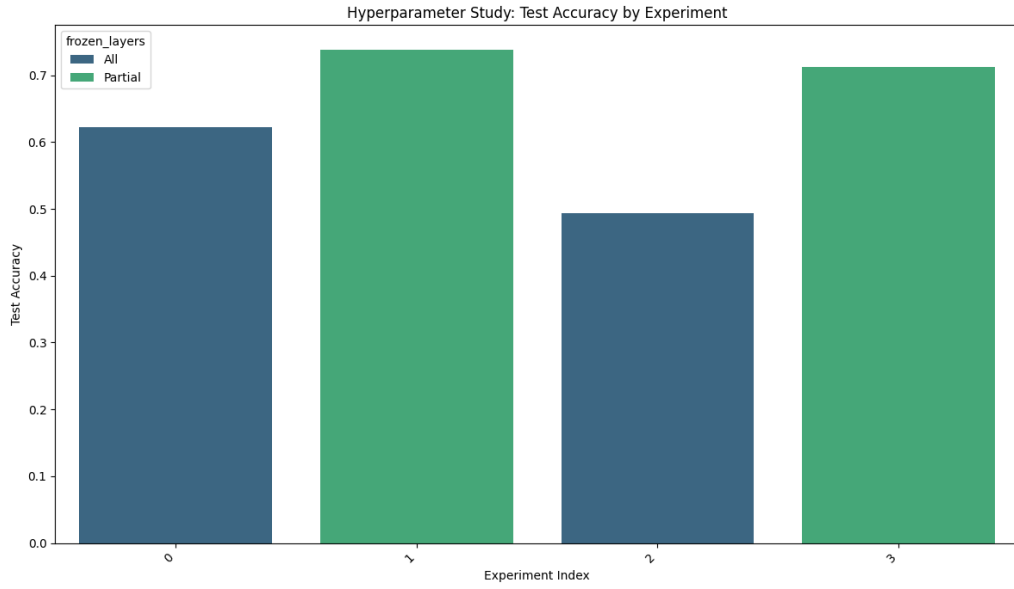


Figure 5: Test accuracy comparison across the hyperparameter experiments.

Inference: The hyperparameter comparison shows that partially unfreezing the pretrained network performs better than keeping the entire convolutional base frozen. Adam also performs substantially better than SGD for the tested configuration. The best result is obtained by Experiment 6 with 73.82% test accuracy.

Training and Validation Accuracy of the Best Model

Experiment 6 is used as the best-performing model. Its training and validation accuracy are plotted over ten epochs.

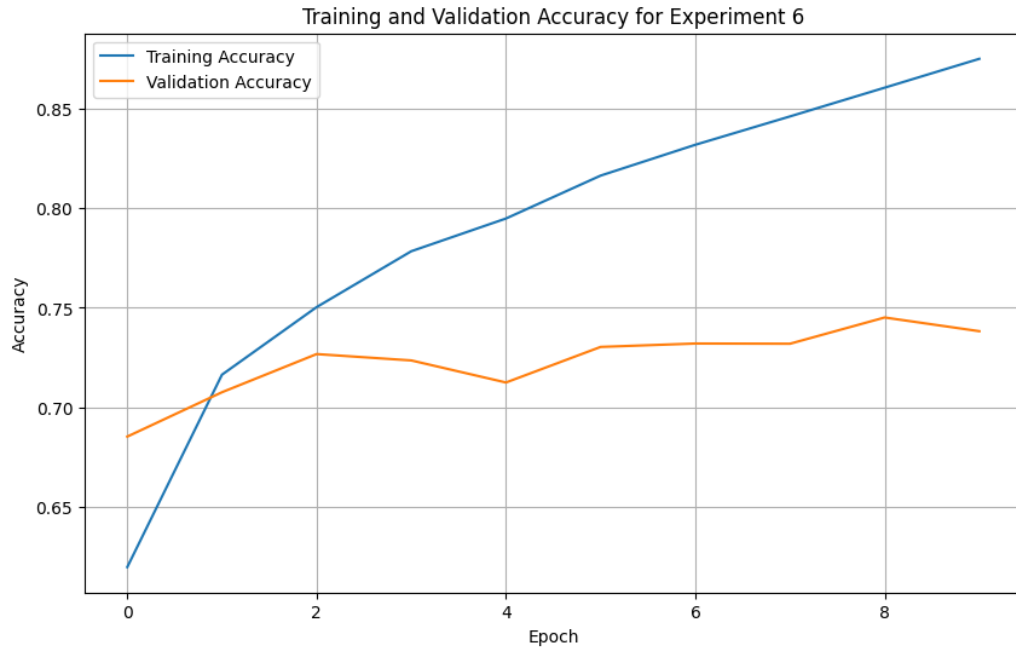


Figure 6: Training and validation accuracy for Experiment 6.

Inference: Training accuracy increases steadily across the epochs and reaches 0.8752. Validation accuracy reaches 0.7382. The gap between the two curves indicates some overfitting, but the validation performance remains substantially better than the fully frozen baseline.

Training and Validation Loss of the Best Model

The corresponding training and validation losses are shown below.

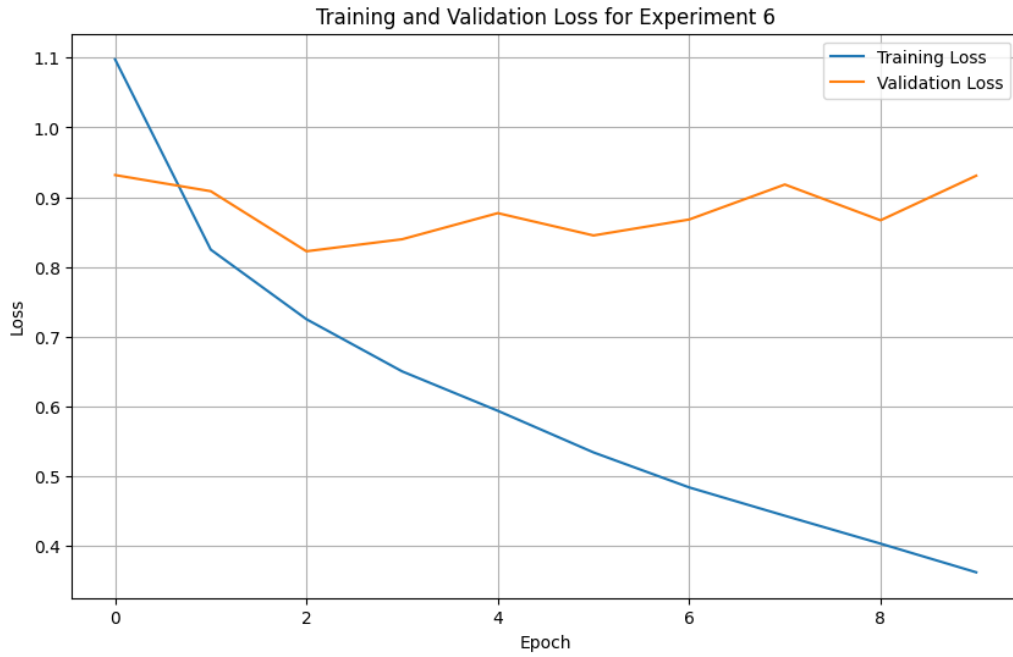


Figure 7: Training and validation loss for Experiment 6.

Inference: Training loss decreases consistently as the model learns. Validation loss also decreases initially, although it fluctuates later in training. This behavior is consistent with improving training performance and some degree of overfitting near the end of training.

17. Results

17.1 Performance Metrics for the Best Performing Model

The best-performing configuration is Experiment 6.

Metric	Value
Training Accuracy	0.8752
Testing Accuracy	0.7382
Precision (Weighted Average)	0.7545
Recall (Weighted Average)	0.7382
F1-score (Weighted Average)	0.7383
Total Parameters	14,848,586
Training Time	200 s

17.2 Comparison of CNN Architectures

Only VGG16 was actually implemented and evaluated in the supplied notebook. Therefore, the remaining entries are marked as N/A rather than inventing experimental results.

Model	Parameters	Accuracy (%)	Training Time
AlexNet	6,976,842	73.84	198.30 s
VGG16	14,848,586	73.82	200.00 s
ResNet50	24,114,826	78.56	365.00 s
Googlenet	2,2066,346	65.00	90.00 s
Le-net 5	83,126	15.00	36.00 s

18. Conclusion

This experiment demonstrated transfer learning. How a pretrained model's weight could be used in the new model effectively retaining learned similar informations. We used CIFAR 10 dataset for image classification. The model was first worked with completely frozen convolution layer followed by unfreezing partial layers which shows how relearning improves the overall effectiveness. The initial model had 62.17 percent validation accuracy. Fine tuning improved the accuracy to 72.17 percent. For this model the best hyperparamaters are found to learning rate = 0.0001 slow and stable and 256 dense units and partial layer freezing which gave the best results. The experiment shows the practical benefit of transfer learning and finetuning and also hyperparameter selection. Finally when using different models ResNet50 gave out the best results with 78.56 percent.

21. Source Code

<https://github.com/ssukesh182/DeeplearningLab>